A quenched QCD lattice implementation using Fortran coarrays
Lab report

Nicolas Lang

Abstract A fortran implementation of quenched QCD using coarrays is presented.

1 Introduction

2 QCD

2.1 The path integral quantization

2.2 Discretization on the lattice

2.3 The QCD action

3 The Metropolis Algorithm

4 Design choices

This section discusses an implementation of the quenched approximation of QCD on the lattice using coarrays for data parallelism.

4.1 Fortran coarrays

Coarrays provide a way to distribute data accross multiple program instances while retaining a concise syntax that abstracts the underlying inter-process or network communication. Formerly an extension of Fortran 95, coarrays are now part of the Fortran 2008 standard. Coarrays are variables with one or several *codimensions* and indices referencing data stored by another program instance, called *image*. *Images* represent copies of the program that run asynchronously and can possess further independent local variables. The coarray entries can be accessed from every image by specifying the respective *coindex* representing the image, where the data is stored. The size of the codimension is determined at runtime from the number of running images. The standard

Bergische Universität Wuppertal

provides for additional methods that enable the user to obtain the number of images and the index of the current image and to introduce a break point to synchronize images. The coarray support in the GNU Fortran compiler is based on the *OpenCoarrays* **opencoarrays** library which in turn is based on the Message Passing Interface (MPI).

4.2 Shortcomings of the current coarray implementation

Lorem ipsum dolor sit amet, consectetur et florabibimus sibet insipitur.

4.3 Overview and general ideas

5 Implementation

5.1 Types

Gauge field

```
type, public :: gauge_field
    integer, allocatable :: indices(:,:,:,:,:,:)
    logical :: tmp
    integer :: dir
    integer :: par
    integer :: x_low, x_up, y_low, y_up, z_low, z_up, t_low, t_up
    integer :: nx, ny, nz, nt
contains

    ! ...

end type gauge_field
```

Field variables

5.2 Operators

Select and shift

```
function select (a, b) result (tmp)
    class(gauge_field), intent(in) :: a
    integer, dimension(2), intent(in) :: b
    type(gauge_field) :: tmp

    call a%copy_shift(tmp, 0, 0)
    tmp%par = b(1)
    tmp%dir = b(2)
    tmp%tmp = .true.
end function
```

```
function shift (a, b) result (tmp)
    class(gauge_field), intent(in) :: a
    integer, dimension(2), intent(in) :: b
    type(gauge_field) :: tmp

    call a%copy_shift(tmp, b(1), b(2))
    tmp%tmp = .true.
end function
```

Arithmetic and logical operators

```
function multiply_fv(a, b) result (res)

    ! ...

    idx = 1
    do d = d_low, d_up
        do i = a%t_low, a%t_up
            do j = a%z_low, a%z_up
                do k = a%y_low, a%y_up
                    do l = a%x_low-1+step_size, a%x_up, step_size
                        if (d == 0) then
                            da = abs(a%dir)
                        else
                            da = d
                        end if
                        if (step_size == 2) then
                            la = l-abs(mod(i+j+k,2)-a%par)
                        else
                            la = l
                        end if

                        res%data(:,:,idx) = multSu3( &
                        data( :,:,a%indices(0,la,k,j,i,da)) &
                        [a%indices(1,la,k,j,i,da)], &
                        (a%dir < 0), &
                        b%data(:,:,idx), &
                        b%conj )
                        idx = idx + 1

                    end do
                end do
            end do
        end do
    end do

end function
```

5.3 The metropolis module

```
function staple(u, p, d)

    ! ...

    call staple%init_zero(s)
    do i = 1, 4
        if (i /= d) then
```

```
            call staple%increment( &
                (u.sel.[p,i]) &
                * (u.shift.[i,1].sel.[p,d]) &
                * (u.shift.[d,1].sel.[p,-i]) &
                + (u.shift.[i,-1].sel.[p,-i]) &
                * (u.shift.[i,-1].sel.[p,d]) &
                * (u.shift.[i,-1].shift.[d,1].sel.[p,i]) &
                )
        end if
    end do

end function staple
```

```
subroutine metro(u, eps, beta, acc)

    ! ...

    ! iterate over both parities and all directions
    do p = 1,2
        do d = 1,4

        ! calc. staple
        stp = staple(u,p,d)
        stp%conj = .true.

        ! generate new link
        call random_number(rnd1)
        call random_number(rnd2)
        if (rnd1 >= 0.5) then
            update_tab(ceiling(rnd2*100))%conj = .true.
        else
            update_tab(ceiling(rnd2*100))%conj = .false.
        end if

        new_link = (u.sel.[p,d]) * update_tab(ceiling(rnd2*100))

        ! calculate action change
        delta_s = -beta / 3.0 * .rtrc.((new_link - (u.sel.[p,d])) * stp)

        ! accept with some prob. dep. on action
        call rnd_field%init_rnd(s)
        call u%update(p, d, new_link, (rnd_field <= exp(-delta_s)), acc_count)
        sync all

        end do
    end do

    acc = real(acc_count) / real((s*8))

end subroutine metro
```

6 Sample measurement